SET B

SRM Institute of Science and Technology
College of Engineering and Technology
School of Computing
SRM Nagar, Kattankulathur – 603203, Chengalpattu District, Tamilnadu
Academic Year: 2025-26 (ODD)

BATCH 1

Test: FJ - II **Date: 26/09/2025**
Course Code & Title: 21CSC201J Data structures and Algorithm **Duration:** 1 hour 40 min
Year & Sem: II Year / III Sem **Max. Marks:** 50

Course Articulation Matrix:

Course Outcome	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO 10	PO 11	PO 12	Program Specific Outcomes		
													PSO - 1	PSO - 2	PSO - 3
CO1	1	2	3	-	-	-	-	-	-	-	-	3	3	-	-
CO2	2	3	3	-	-	-	-	-	-	-	-	3	3	-	-
CO3	2	3	3	-	-	-	-	-	-	-	-	3	3	-	-
CO4	2	3	3	-	-	-	-	-	-	-	-	3	3	-	-
CO5	3	2	3	-	-	-	-	-	-	-	-	3	3	-	-

Part – A (4 x 2 = 8 Marks) Instructions: Answer all

Q. No	Question	Marks	BL	CO	PO	PI Code
1.	A singly linked list contains the elements: 10 → 20 → 30 → 40 → NULL What will be the output of the following pseudocode? ptr ← head while ptr ≠ NULL do print(ptr.data) ptr ← ptr.next.next end while A. 10 20 30 40 B. 40 30 20 10 C. 10 20 30 D. 10 30 Ans: D. 10,30	2	L3	2	2	2.1.1
2.	In a Josephus problem with n = 9 people standing in a circle and every third person (k=3) is eliminated, what is the position of the last survivor (1-based indexing)? Ans: Position 1	2	L3	2	2	2.1.1
3.	An array-based list has size = 5. Initially, it	2	L3	3	2	2.1.1

	contains: [10, 40, 30, 20, -] If we insert 25 at position 2(0-based index), and delete element at third position what will the array look like? A. [10, 40, 30, 25] B. [10, 40, 25, 30] C. [10, 40, 25, 20] D. [10, 40, 30, 20] Ans: C. [10, 40, 25, 20]					
4.	What is the postfix expression of the following infix expression: A + (B * C - (D / E ^ F) * G) * H Ans: A B C * D E F ^ / G * - H * +	2	L3	3	2	2.1.1
Part – B (2 x 5 = 10 Marks) Instructions: Answer all						
5	Write a pseudocode to traverse a singly linked list and count the number of even integer elements present in it. Ans: CountEvenElements(head) Input: head (pointer to the first node of the singly linked list) Output: count of even integer elements count = 0 ptr = head while ptr ≠ NULL do if (ptr ← data % 2 = 0) then count = count + 1 ptr = ptr ← next return count	5	L2	2	2	2.1.2
6	In a CPU scheduling system, a deque is used to allow insertion and deletion from both ends. Suppose the deque initially has the processes: **P1, P2, P3, P4** A. Perform the following operations and show the result after each step: DeleteFront(), InsertRear(P5), InsertFront(P6), DeleteRear(), InsertRear(P7) B. Write pseudocode for InsertRear() and DeleteFront().	5	L2	3	2	2.1.2

	Ans: a) Initial state of deque: Front → [P1, P2, P3, P4] ← Rear DeleteFront(): [P2, P3, P4] InsertRear(P5): [P2, P3, P4, P5] InsertFront(P6): [P6, P2, P3, P4, P5] DeleteRear(): [P6, P2, P3, P4] InsertRear(P7): [P6, P2, P3, P4, P7] Final deque after all operations: Front → [P6, P2, P3, P4, P7] ← Rear b) InsertRear(x) InsertRear(x) if deque is full then print "Overflow" return rear ← (rear + 1) mod capacity deque[rear] ← x DeleteFront() if deque is empty then print "Underflow" return item ← deque[front] if front = rear then front ← -1 rear ← -1 else front ← (front + 1) mod capacity end if return item					
Part – C (4 x 8 = 32 Marks) **Instructions:** • **Answer a total of FOUR (4) questions.** • **From Questions 7, 8, and 9, answer ANY TWO.** • **From Questions 10, 11, and 12, answer ANY TWO.** • **All questions carry equal marks.**						
7	A group of 20 friends are planning a secret gift exchange game. They sit in a circle, numbered 1 to 20. Starting from friend number 1, every 3rd person is eliminated from the circle until only one friend remains, who will receive a special surprise gift.	8	L3	2	2	2.1.3

1. Determine the position of the friend who will get the special gift using the Josephus Problem concept. Show all calculations or reasoning. **(4 Marks)**
2. If the elimination counts changes from every 3rd person to every 4th person, which position will get the gift? Explain the method you use. **(2 Marks)**
3. Suggest a modification in the seating or elimination rule so that a specific friend of your choice (say, friend number 7) is guaranteed to get the gift. Explain your reasoning. **(2 Marks)**

Ans:

(1)

Elimination order (by rounds):

1. **Round 1:** 3, 6, 9, 12, 15, 18
   Remaining after Round 1: 1, 2, 4, 5, 7, 8, 10, 11, 13, 14, 16, 17, 19, 20
2. **Round 2:** 1, 5, 10, 14, 19
   Remaining after Round 2: 2, 4, 7, 8, 11, 13, 16, 17, 20
3. **Round 3:** 4, 11, 17
   Remaining after Round 3: 2, 7, 8, 13, 16, 20
4. **Round 4:** 7, 16
   Remaining after Round 4: 2, 8, 13, 20
5. **Round 5:** 8
   Remaining after Round 5: 2, 13, 20
6. **Round 6:** 2
   Remaining after Round 6: 13, 20
7. **Round 7:** 13
   Last remaining (winner): 20

Putting the full elimination sequence in order:
3, 6, 9, 12, 15, 18, 1, 5, 10, 14, 19, 4, 11, 17, 7, 16, 8, 2, 13, 20.

Answer: Friend number 20 is the survivor and so gets the special gift.

(2)
Eliminate every 4th person, counting continuously around the circle starting from person 1.

	Elimination shown round-by-round **Round 1 (counting from 1):** 4, 8, 12, 16, 20 Remaining after Round 1: 1, 2, 3, 5, 6, 7, 9, 10, 11, 13, 14, 15, 17, 18, 19 **Round 2 (continue counting from after 20 → back to 1):** 5, 10, 15, 1, 7, 14, 2, 11, 19 Remaining after Round 2: 3, 6, 9, 13, 17, 18 **Round 3 (continue counting from after 19 → 3):** 13, 6, 3, 9 Remaining after Round 3: 17, 18 **Round 4:** 18 **Last remaining (winner): 17** **Full elimination sequence (in order):** 4, 8, 12, 16, 20, 5, 10, 15, 1, 7, 14, 2, 11, 19, 13, 6, 3, 9, 18, 17. **Friend number 17** will receive the special gift when every 4th person is eliminated. (3) **For example, if we want that the person in 17th place should get gift, then we can fix the elimination count as 15.**					

8	Given k sorted linked lists of different sizes, write an algorithm and illustrate with an example to merge them into a single list while maintaining their sorted order.
	
	Ans:
	struct Node {
	int data;
	struct Node* next;
	};
	
	CreateNode(value)
	{
	Struct Node node;
	node ← malloc(sizeof(Node))
	node.data ← value
	node.next ← NULL
	return node
		8	L3	2	2	2.1.3

	}
	MergeTwoLists(l1, l2)
	{
	// Create a dummy node to simplify pointer operations
	dummy ← CreateNode(0)
	tail ← dummy
	
	while l1 ≠ NULL and l2 ≠ NULL
	{
	if l1.data ≤ l2.data then
	tail.next ← l1
	l1 ← l1.next
	else
	tail.next ← l2
	l2 ← l2.next
	tail ← tail.next
	}
	// Attach the remaining nodes
	if l1 ≠ NULL
	tail.next ← l1
	else
	tail.next ← l2
	
	return dummy.next
	}
	
	MergeKLists(lists[0…k-1])
	{
	if k = 0 then
	return NULL
	
	result ← lists[0]
	
	for i = 1 to k-1
	result ← MergeTwoLists(result, lists[i])
	
	return result
	}					

9	Consider the following sparse matrix A of size 4 × 5: [0 0 3 0 4]					

[0 0 0 0 0]
[2 0 0 0 1]
[0 5 0 0 0]

(a) Write the complete triplet representation of matrix S, including the header. (**4 Marks**)

(b) Write pseudocode for a function SparseToTriplet(mat, rows, cols) that:

- Accepts a 2D array mat of size rows × cols,
- Scans the matrix in row-major order,
- Constructs and returns the triplet representation as a list of tuples. (**4 Marks)**

Ans:

a)

Triplets:

Header: (4, 5, 5)

(0, 2, 3)
(0, 4, 4)
(2, 0, 2)
(2, 4, 1)
(3, 1, 5)

b)

```
SparseToTriplet(mat, rows, cols)
{
   // Initialize list of triplets
   triplet_list ← []

   // Count non-zero elements
   count ← 0

   // Scan matrix in row-major order
   for i from 0 to rows-1
      for j from 0 to cols-1
         if mat[i][j] ≠ 0
            triplet_list.append((i, j, mat[i][j]))
            count ← count + 1
```

	// Prepend header (rows, cols, count)
	triplet_list.insert(0, (rows, cols, count))
	
	return triplet_list
	}					

10	A stack is implemented using an array of size 5. The following operations are defined: • TopAndPop(): Returns the top element of the stack and then removes it. • PopAndTop(): Removes the top element of the stack and then returns the new top element. Initially, the stack is empty. The following sequence of push operations is performed: Push(10), Push (20), Push(30), Push (40) Answer the following: a) Write a Pseudocode to show the output of TopAndPop()? Show the stack after this operation. (**3 marks**) b) Write a Pseudocode to show the output of PopAndTop()? Show the stack after this operation. **(3 marks**) c) Clearly explain the difference between these two functions using this example. (**2 marks**) Ans: a) TopAndPop() TopAndPop(stack, top) { if top < 0 { print "Stack Underflow" return NULL	8	L3	3	2	2.1.3

	}
	
	value ← stack[top] // get top element
	top ← top - 1 // remove it
	return value
	}
	
	b)
	PopAndTop()
	PopAndTop(stack, top)
	{
	if top < 0
	{
	print "Stack Underflow"
	return NULL
	}
	
	top ← top - 1 // remove top element
	
	if top < 0
	{
	print "Stack is empty"
	return NULL
	}
	
	return stack[top] // return new top element
	}
	
	C)
	Difference between TopandPop and PopandTop:
	
	TopAndPop() → returns 40, stack = [10,20,30]
	
	PopAndTop() → returns 30 (new top after removing 40), stack = [10,20,30]
	
	TopAndPop() focuses on what was removed.
	
	PopAndTop() focuses on what is on top after removal.					

11	Convert the following infix expression to its					

equivalent postfix form using a stack.

Infix Expression:

A + B * C - (D / E)

a. Show the stack contents and output string after processing each symbol.(**4 Marks**)

b. Write pseudocode for a function InfixToPostfix(infix) that converts any valid infix expression into postfix notation.(**4 Marks**)

Ans:

a)

Symbol	Action	Stack (top on right)	Output
A	Operand → add to output	[]	A
+	Operator → push	+	A
B	Operand → add to output	+	AB
*	Higher precedence than top → push	+, *	AB
C	Operand → add to output	+, *	ABC
-	Pop * and + (≥ precedence) → push -	-	ABC*+
(	Push '('	-, (	ABC*+
D	Operand → add to output	-, (	ABC*+D
/	Operator → push	-, (, /	ABC*+D
E	Operand → add to output	-, (, /	ABC*+DE
)	Pop until '(' → pop /	-	ABC*+DE/
End	Pop remaining operators	[]	ABC*+DE/-

Final Postfix Expression: ABC*+DE/-

b)

```
InfixToPostfix(infix)
{
   stack ← empty
   output ← empty string

   for each symbol in infix
{
      if symbol is operand
         output ← output + symbol
      else if symbol is '('
         push stack, '('
      else if symbol is ')'
         while top of stack ≠ '('
```

	output ← output + pop(stack)
	pop(stack) // remove '('
	else: // operator
	while stack is not empty and precedence(top(stack)) ≥ precedence(symbol)
	output ← output + pop(stack)
	push(stack, symbol)
	}
	
	while stack is not empty
	output ← output + pop(stack)
	
	return output
	}					

12	A compiler maintains a call stack for nested and recursive function calls. Explain how a stack ADT helps in storing activation records (function parameters, return addresses). Illustrate the working to compute Fib(7), the seventh number in Fibonacci series. Ans: **Call Stack for Computing Fib(7) Using Recursion** **Function Definition:** `int Fib(int n) { if (n <= 1) return n; return Fib(n-1) + Fib(n-2); }` **Step-by-Step Stack Operations** **Notation:** Each stack frame shows `Fib(n)` being called.	8	L3	3	2	2.1.3

Step	Stack (Top → Bottom)	Action / Result
1	Fib(7)	Initial call, push Fib(7)
2	Fib(6)	
Fib(7)	Call Fib(6), push Fib(6)	
3	Fib(5)	
Fib(6)		
Fib(7)	Call Fib(5), push Fib(5)	
4	Fib(4)	
Fib(5)		
Fib(6)		
Fib(7)	Call Fib(4), push Fib(4)	
5	Fib(3)	
Fib(4)		
Fib(5)		
Fib(6)		
Fib(7)	Call Fib(3), push Fib(3)	
6	Fib(2)	
Fib(3)		
Fib(4)		

Step	Stack (Top → Bottom)	Action / Result
Fib(5)		
Fib(6)		
Fib(7)	Call Fib(2), push Fib(2)	
7	Fib(1)	
Fib(2)		
Fib(3)		
Fib(4)		
Fib(5)		
Fib(6)		
Fib(7)	Call Fib(1), base case → 1	
8	Fib(2)	
Fib(3)		
Fib(4)		
Fib(5)		
Fib(6)		
Fib(7)	Pop Fib(1), return 1	
9	Fib(0)	
Fib(2)		
Fib(3)		
Fib(4)		
Fib(5)		
Fib(6)		
Fib(7)	Call Fib(0), base case → 0	
10	Fib(2)	
Fib(3)		
Fib(4)		
Fib(5)		
Fib(6)		
Step	Stack (Top → Bottom)	Action / Result
Fib(7)	Pop Fib(0), return 0	
11	Fib(3)	
Fib(4)		
Fib(5)		
Fib(6)		
Fib(7)	Compute Fib(2)=1, pop Fib(2)	
12	Fib(4)	
Fib(5)		
Fib(6)		
Fib(7)	Compute Fib(3)=2, pop Fib(3)	
13	Fib(5)	
Fib(6)		
Fib(7)	Compute Fib(4)=3, pop Fib(4)	
14	Fib(6)	
Fib(7)	Compute Fib(5)=5, pop Fib(5)	
15	Fib(7)	Compute Fib(6)=8, pop Fib(6)
16	(empty)	Compute Fib(7)=13, pop Fib(7)

Final Result:

```
Fib(7) = 13
Stack is empty
```

Key Notes:

1. Each **push** corresponds to a function call.
2. Each **pop** occurs when the function returns.
3. The stack ensures **LIFO order**, so recursion works correctly.
4. Activation records store **n, return address, and temporary results**.

Course Outcome (CO) and Bloom's level (BL) Coverage in Questions

